

AI-Powered Interactive Tool

Flam — Frontend Internship Assignment

Candidate Reference Guide

Quick Facts

Time budget: ~8 hours, hard cap · Stack: **React (hooks) + any LLM API**

Pick **one** project idea · **Must not be a chatbot** — structured data only

ASSIGNMENT LINK :  Frontend_Internship_Assignment.pdf

1. Objective

Build a small React app that takes free-form text input, sends it to an AI model, and turns the result into an interactive tool — not a chat window. Calling the model is the easy part; the assignment is really about how you turn unpredictable AI output into reliable UI, and how gracefully you handle it when the model gets things wrong.

What you'll practice:

- Calling a real LLM API and requesting structured (JSON) output instead of free text
- Parsing and validating unpredictable model output before it ever reaches your UI
- Designing real React state and interactive components around that structured data
- Handling every realistic failure mode: malformed JSON, wrong shape, empty, slow, or failed responses
- Keeping an API key out of the browser by routing calls through a small backend

2. Technical Requirements

- **React with hooks, functional components** — required. TypeScript is optional and not graded.
- **A free-form text input** — the only way the user gets information into the app.
- **A real LLM API, any provider** — Gemini, Groq, and OpenRouter have free tiers; OpenAI and Anthropic are cheap; or run a model locally for free with Ollama. Smaller local models are less consistent at structured output, so your failure handling matters even more if you go that route.
- **A small backend or serverless function** — to hold your API key. Don't ship the key in the browser — this is explicitly checked.
- **Any tooling that helps** — Vite/Next, a CSS framework, AI SDKs. If an SDK handles structured output or streaming for you, be ready to explain what it's actually doing.
- **No auth needed** — and deployment is preferred but optional.

3. Project Features — Pick One

Same requirements apply to each — choose whichever you'd enjoy building:

- **Study assistant** — user pastes notes or a topic; the AI generates flashcards or a quiz; the app lets them flip through cards, take the quiz, and re-test wrong answers.
- **Fridge-to-recipe** — user lists ingredients; the AI returns a recipe; the app shows steps to check off, scalable servings, and ingredient swaps.
- **Trip planner** — user describes a trip; the AI returns a day-by-day itinerary; the app lets them expand, remove, and reorder stops.

The one firm rule

It can't be a chatbot. The AI must return structured data (e.g. JSON) that your code parses and renders as interactive components. Printing the model's raw text in a chat box does not meet the requirement.

4. Step-by-Step Implementation Guide

1. **Pick one idea and design the data shape first.** Sketch the exact JSON shape you want back (e.g. an array of flashcard objects, a recipe with steps and swaps, an itinerary of days and stops) before writing a single prompt. This is what turns the project from “print AI text” into real structured-data UI.
2. **Scaffold the React app.** Vite or Next, hooks, and a free-form text input wired to local state.
3. **Set up the backend proxy.** A small backend or serverless function that holds your API key and forwards the prompt to your chosen LLM provider.

4. **Write a strict prompt.** Ask the model to return exactly the shape from Step 1, JSON only. Test it directly, outside your UI, so you've actually seen what real — and malformed — output looks like.
5. **Parse and validate before rendering.** Check the response is valid JSON and matches the expected shape; anything else should route to an error state, never straight to the UI.
6. **Build the interactive UI.** Render the parsed data as real components — flip cards, checkable steps, reorderable stops — driven by React state, not by re-printing the model's text.
7. **Handle every failure mode explicitly.** Malformed JSON, wrong shape, empty, slow, and failed responses each need a visible state — no crashes, and no silent hangs.
8. **Add loading, error, and empty states, and check mobile.** Confirm the layout holds up at a mobile-width viewport, not just on your desktop screen.
9. **Layer in stretch goals only once the core is solid.** See Section 9 — these are optional, and a clean core beats a pile of half-working extras.
10. **Write the README and record a demo.** Setup, usage, an honest AI-usage note, known limitations, and time spent — then commit in small, meaningful steps.

5. Project Structure

```
flam-frontend-assignment/
├── src/
│   ├── components/
│   │   ├── PromptInput.tsx      # free-form text input
│   │   ├── ResultView.tsx      # routes parsed data to the right UI
│   │   ├── FlashcardDeck.tsx   # example: study-assistant view
│   │   ├── ErrorState.tsx      # shared error / retry UI
│   │   └── LoadingState.tsx
│   ├── lib/
│   │   ├── api.ts              # calls your backend proxy, never the LLM directly
│   │   └── validateResult.ts    # checks the shape before rendering
│   ├── types/
│   │   └── result.ts           # the structured shape you designed in Step 1
│   └── App.tsx
├── server/
│   └── generate.ts             # holds the API key, calls the LLM, returns JSON
├── .env.example
├── README.md
└── package.json
```

- **lib/api.ts** — the only place the frontend talks to your backend; the LLM call itself never happens in the browser.
- **lib/validateResult.ts** — kept separate so shape-checking is easy to point to and reason about on its own.
- **components/ErrorState.tsx + LoadingState.tsx** — shared across every view, so failure handling is consistent, not re-invented per screen.

6. Example Code Snippets

Requesting a strict structured shape

```
const prompt = `Return ONLY valid JSON matching this shape, no prose:
{ "cards": [{ "question": string, "answer": string }] }
Topic: ${userInput}`;
```

Being explicit and strict about the shape — and asking for JSON only, no prose — is what makes parsing reliable instead of hopeful.

Defensive parsing before rendering

```
function parseResult(raw) {
  try {
    const data = JSON.parse(raw);
    if (!Array.isArray(data.cards)) return null;
    return data;
  } catch {
    return null;
  }
}
```

A null here should route to your error state — never to a blank render or a crash.

Guarding against a stale response

```
const requestId = useRef(0);
async function generate(input) {
  const id = ++requestId.current;
  const result = await callApi(input);
  if (id !== requestId.current) return; // a newer request has since started
  setResult(result);
}
```

Without this guard, a slow first request can resolve after a faster second one and silently overwrite the correct, newer result.

7. Error Handling Guidelines

Most of the signal in this assignment is here — handling failure well is what separates people who've built AI features from those who haven't.

- **Malformed JSON** — catch the parse failure, show a clear error state, offer a retry.
- **Wrong shape** — valid JSON but missing fields; validate structurally, don't assume, and fall back to an error state.
- **Empty response** — treat it as a failure, not as an empty-but-valid result.
- **Slow response** — show a loading state; don't let a request silently hang forever.
- **Failed request** — show an error, not a crash; offer a retry.
- **Stale responses** — a newer request's result must never be overwritten by an older, slower one resolving late.

8. AI Tools & Original Work

Using AI tools — Copilot, Cursor, Claude, ChatGPT, whatever you normally use — is expected. Add a short, honest README note on what you used AI for; being upfront about it counts in your favor.

The work itself must be your own. Referencing docs, tutorials, and AI assistants is fine; submitting an existing project, a tutorial result, or someone else's code as your own is not. You'll explain and extend your code live in the interview, so anything you didn't build yourself will be clear.

9. Stretch Goals (Optional)

- Let the AI return different kinds of blocks (a card, a chart, a checklist) and render each appropriately
- Stream the result as it generates
- A refinement loop — follow-up prompts that edit the existing result instead of regenerating from scratch
- Save and reload sessions
- Polish: animation, dark mode, keyboard navigation

10. Evaluation Criteria

- **React & frontend architecture** — 25%
- **AI integration & data handling** — 25%
- **Handling bad AI output** — 20%
- **UI/UX & product sense** — 15%
- **Communication & understanding** — 15%

A clean, solid core beats a pile of half-working features.

11. If You Move Forward: The Interview

Expect to demo your app, walk through your code, review a short AI-generated snippet, fix a bug the interviewers introduce, and add a small feature — live. Don't ship code you don't understand.

12. Submission Guidelines

1. **A public GitHub repo** (or private with access) — small, meaningful commits beat one large one.
2. **A README** covering setup, usage, an AI-usage note, known limitations, and time spent.
3. **A short screen recording** showing the app working, plus instructions to run it locally — `npm install` && `npm start` should just work.

Time: aim for ~8 hours total; don't spend more than 8 hours of actual work. If you run out, stop and note what you'd do next.

Quick FAQ

TypeScript required? **No.**

AI tools/SDKs allowed? **Yes** — just understand your code.

Which model? **It** doesn't affect your score.

Auth needed? **No.**

Deploy required? **Preferred.**

No budget for API credits? **Use a free tier or run Ollama locally.**